Two-mass Spring System

Two masses m_1 and m_2 connected by a spring with constant k lying on a frictionless floor. Let $p_1(t)$ and $p_2(t)$ be the displacement of m_1 and m_2 from their initial references, respectively. By Newton's second law, the ODEs describing the dynamics are:

$$m_1\ddot{p}_1(t) = k(p_2(t) - p_1(t)), \quad m_2\ddot{p}_2(t) = -k(p_2(t) - p_1(t)), \quad t \geq 0.$$

It is assumed that the two masses are at rest initially, but at least one displacement is non-zero. For the Grader problem, we are interested in the total perturbation $p_1(t) + p_2(t)$.

Problem

1. Express the ODEs into a state-space form $\dot{x} = Ax$ where the state vector is defined as $x = (p_1, p_2, \dot{p}_1, \dot{p}_2)$. Provide the resulting matrix A in the **sum_twomass_spring** function.
2. Write a Python function that simulates the response of $p_1(t) + p_2(t)$ at different times when the list of $t = (t_1, t_2, \ldots, t_K)$ is given. The function takes the masses m_1, m_2, the spring constant (k), an array t, and an initial condition x_0. It should returns an array of sum of two masses' positions at each time. Then, submit this function to Grader.

Also observe the result when $m_1 = m_2$. What do you see?

Implementation

Goal: Complete the function below and submit only the function to the Grader.

```python
import numpy as np
# FYI: you can also import scipy

def sim_twomass_spring(k, m1, m2, t, x0):
    ...
    return sumpos

if __name__ == "__main__":
    # write your test section here
    np.random.seed(67)
    k, m1, m2 = 5, 1, 1
    # Random initial displacement for p1, rest are zero
    x0 = np.array([np.random.randint(-5, 5), 0, 0, 0])
    t_points = np.array([1, 5, 50, 100, 1000, 2000])

    sumpos = sim_twomass_spring(k, m1, m2, t_points, x0)
    print(f"sumpos: {sumpos}")
```

Constraints

Input:

- $m_1, m_2, k > 0$.
- $t \in \mathbb{R}^K$ with values in range $[1, 2000]$.
- $x_0 \in \mathbb{R}^4$.

Output:

- `sumpos`: A NumPy array of size K representing $p_1(t) + p_2(t)$.

Sample Output

Note: Expected output from the parameters in the `if __name__ == "__main__"` block:

```
sumpos: [-2. -2. -2. -2. -2. -2.]
```